MEMBER 2 • SERVICE PACKAGES, CUSTOM EXCEPTIONS & CONCURRENCY

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (YOU OWN THE CORE EXCEPTION HIERARCHY!)

📦 **Packages:** `com.cricbid.service`, `strategy`, `exception`

🔌 **Interfaces:** `BiddingStrategy` (`AggressiveBot`, `BalancedBot`, `MoneyballBot`)

📚 **Java Libraries:** `ReentrantLock`, `ScheduledExecutorService`, `javax.swing.Timer`, `EnumMap`

🛡️ **Exception Handling:** `CricBidException` + 6 custom subclasses, `try-finally` unlock, JDBC `rollback()`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Continuing from Member 1, I'll cover our `service`, `strategy`, and `exception` packages, highlighting how we use Java concurrency libraries, interfaces, and custom exception handling.

In `com.cricbid.service`, `AuctionService` manages the player pool, highest bid, and countdown timer using **Java libraries** like `javax.swing.Timer`, `java.util.EnumMap`, and `java.util.concurrent.locks.ReentrantLock`. Whenever a bid arrives, execution enters a critical section guarded by `auctionLock`, with `unlock()` inside a `finally` block so a deadlock can never occur. For **interfaces**, our `com.cricbid.strategy` package defines the `BiddingStrategy` interface implemented by `AggressiveBot`, `BalancedBot`, and `MoneyballBot`.

Inside the lock, `RuleValidator` enforces **custom exception handling** through our `com.cricbid.exception` package. Extending our checked base class `CricBidException`, it throws specific exceptions— `InvalidBidException`, `SquadFullException`, `OverseasLimitException`, `BudgetExceededException`, and `ReserveViolationException`—whenever any IPL rule is broken. When a round ends, `executeSaleTransaction()` and `TradeService.executeTrade()` use `setAutoCommit(false)` inside `try-catch-finally`, calling `commit()` if all four SQL updates succeed or `rollback()` on `SQLException`.

For Review 2, we defaulted bots to OFF, added `placeHumanBid()` so multiple human laptops can outbid each other, and made `ReserveViolationException` dynamic: requiring minimum 8 players in Quick Mode and 18 in Full Mode."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
Checked Exception (`extends Exception`)	A custom error class (`CricBidException`) where Java's compiler **forces** calling code to write `try-catch` or `throws`, guaranteeing broken auction rules are never ignored!
`throw` vs `throws`	`throw new BudgetExceededException(...)` blows the referee whistle inside the method; `throws BudgetExceededException` on the method header warns callers that this method might blow that whistle.

`ReentrantLock` (`java.util.concurrent.locks`)	A thread traffic gate (`auctionLock.lock()`). If Laptop 2 and Laptop 3 bid at the exact same millisecond, only 1 thread enters first so two teams never get the same price!
`try / catch / finally`	`try` = normal code; `catch` = recover from error (`conn.rollback()`); `finally` = cleanup code that **ALWAYS runs 100% of the time** (`auctionLock.unlock()`).
`EnumMap` (`java.util` library)	A super-fast Java `Map` library designed specifically for enum keys (`AGGRESSIVE` , `BALANCED` , `MONEYBALL`) to look up `BiddingStrategy` implementations.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `com.cricbid.exception` Package & `RuleValidator.java` — Custom Exceptions (5 Marks Star!)

📂 Open: src/main/java/com/cricbid/exception/CricBidException.java & service/RuleValidator.java (Lines 54–117)

```
// Package com.cricbid.exception — Base Checked Exception
public class CricBidException extends Exception {
    public CricBidException(String message) { super(message); }
}

// Inside RuleValidator.java (Lines 54–116) — Throws 5 Custom Checked Subclasses!
public void validateBid(Team team, Player player, double proposedAmount,
                        Integer currentLeadingTeamId, double currentPrice, boolean
hasExistingBids)
        throws SquadFullException, OverseasLimitException, InvalidBidException,
               BudgetExceededException, ReserveViolationException {

    if (currentLeadingTeamId != null && currentLeadingTeamId.equals(team.getTeamId()))
        throw new InvalidBidException("Team already holds the highest bid.");
    if (team.getSquadSize() >= 25)
        throw new SquadFullException("Reached maximum squad limit of 25 players.");
    if (player.isOverseas() && team.getOverseasCount() >= 8)
        throw new OverseasLimitException("Reached maximum limit of 8 overseas players.");
    if (proposedAmount < minRequiredBid - 0.001)
        throw new InvalidBidException("Bid is below minimum required bid.");
    if (proposedAmount > team.getPurseRemaining())
        throw new BudgetExceededException("Insufficient remaining purse.");

    int activeMinSquad = getActiveMinSquadSize(); // 8 in Quick Mode, 18 in Full Mode (NEW in
Review 2!)
    int remainingSlotsNeededToMin = Math.max(0, activeMinSquad - (team.getSquadSize() + 1));
    double requiredReserveFunds = remainingSlotsNeededToMin * 0.20;
    if (team.getPurseRemaining() - proposedAmount < requiredReserveFunds - 0.001)
        throw new ReserveViolationException(...);
}
```

Exact Words to Say to Sir:

"Sir, instead of using generic errors or returning `false`, we created a dedicated `com.cricbid.exception` package with `CricBidException` extending `Exception` and 6 specific subclasses. When any rule fails—like our Reserve Rule saving `0.20 Cr` per remaining mandatory slot (8 slots in Quick Mode, 18 in Full Mode)— `RuleValidator` throws the exact exception so the offending laptop sees the exact reason in a popup."

File 2: `AuctionService.java` — `ReentrantLock` Library & `try-finally` Safety

📂 Open: src/main/java/com/cricbid/service/AuctionService.java (Lines 58–61 & Lines 193–262)

```
private final Map<BotStrategyType, BiddingStrategy> strategyMap = new EnumMap<>
(BotStrategyType.class);
private final ReentrantLock auctionLock = new ReentrantLock(); // java.util.concurrent.locks
library!
...
public boolean placeHumanBid(Team team, double amount) throws CricBidException, SQLException {
    return placeBid(team, amount, true); // NEW in Review 2: lets multiple human laptops outbid
each other
}

public boolean placeBid(Team team, double amount, boolean isHumanBidder) throws CricBidException,
SQLException {
    auctionLock.lock();                                          // Lock critical section
    try {
        ruleValidator.validateBid(team, currentPlayer, amount, leadingTeamId, currentPrice,
roundHasBids);
        bidDAO.recordBid(bid);
        this.currentPrice = amount;
        this.currentLeadingTeam = team;
        this.secondsRemaining = roundDurationSec;                // Reset countdown timer!
        auctionSubject.notifyBidPlaced(bid, team, currentPlayer);
        return true;
    } finally {
        auctionLock.unlock();                                    // ALWAYS unlocks even if
exception thrown!
    }
}
```

File 2 (Continued): `AuctionService.java` — JDBC Transaction & `rollback()` Exception Handling

📂 Stay in: AuctionService.java (Scroll to Lines 280–352: `executeSaleTransaction`)

```
conn = DBConnection.getInstance().getConnection();
conn.setAutoCommit(false); // Start atomic transaction
try {
    // 1. UPDATE teams SET purse_remaining = ?      (Deduct purse)
    // 2. INSERT INTO squads ...                    (Assign player)
    // 3. UPDATE players SET status = 'SOLD'        (Mark SOLD)
    // 4. INSERT INTO auction_log ...               (Audit log)
    conn.commit();         // All 4 succeeded -> Save permanently!
} catch (SQLException ex) {
    if (conn != null) {
        try {
            conn.rollback(); // Any step failed -> Undo EVERYTHING!
        } catch (SQLException rollbackEx) {
            ex.addSuppressed(rollbackEx); // Java exception chaining!
        }
    }
    throw ex;
} finally {
    conn.setAutoCommit(originalAutoCommit);
    conn.close();
}
```

- **Why this scores full marks on Exception Handling:** Uses `try`, `catch` (`conn.rollback()` + `ex.addSuppressed()`), and `finally` (`setAutoCommit` + `close()`) so partial database corruption is impossible! (`TradeService.executeTrade()` uses the exact same pattern.)

File 3: `BiddingStrategy.java` — Interface in `com.cricbid.strategy`

📂 Open: src/main/java/com/cricbid/strategy/BiddingStrategy.java

```
package com.cricbid.strategy;

public interface BiddingStrategy {
    double decideBid(Team botTeam, Player player, double currentBid, boolean hasExistingBids);
    String getStrategyName();
}
```

- **Strategy Interface:** Implemented by `AggressiveBot`, `BalancedBot`, and `MoneyballBot`, stored inside `java.util.EnumMap` in `AuctionService`.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: What is the difference between Checked and Unchecked Exceptions, and which did you use?

Answer: "Checked exceptions extend `Exception` and are verified at compile time; unchecked exceptions extend `RuntimeException`. All 7 classes in `com.cricbid.exception` are Checked Exceptions so Java forces `AuctionService` and `AuctionServer` to catch every rule violation."

Q2: Why is `auctionLock.unlock()` inside a `finally` block instead of at the end of `try`?

Answer: "If `validateBid()` throws `BudgetExceededException`, execution jumps out of `try` immediately. Without `finally`, the lock would stay locked forever and freeze all future bids in a deadlock!"

Q3: Which Java concurrency libraries did you use in `AuctionService`?

Answer: "We used `java.util.concurrent.locks.ReentrantLock` for thread-safe bid validation, `java.util.concurrent.ScheduledExecutorService` for optional bot scheduling, and `javax.swing.Timer` for the 1-second round countdown."